TOOLKIT

Master Rule Book

Every rule from all 18 days, grouped by theme.

Every rule from all 18 daily rule books, grouped by theme. **Page 1** is the short list: the rules that came back again and again. **Sections 1–13** hold every rule, each tagged with the day it was taught (e.g. `W1D1`); open that day's rule book for the exercise steps and examples.

The rules on one page

If you remember nothing else from the six weeks, remember these.

#	Rule	See
1	**Save a copy in Drive, run cells top to bottom, and run STEP 0 again after any restart**	§1
2	**Never paste real NIGCOMSAT data into an AI tool**, and never put raw rows on a projector when outsiders are present	§1 §12
3	**Declare the rhythm**: `"h"` hourly, `"D"` daily, `"W-MON"` weekly, `"MS"` monthly. Never the old capital `"H"`	§2
4	**Threshold on the chart, finding in the title.** Your oga reads the title, not the axes	§3
5	**Past only.** Trailing windows, `shift(+n)`, and every feature passes the 14:00 test	§4 §9
6	**The process picks the period; the correlogram confirms it**	§5
7	**Limits come from a quiet reference period, one set per operating state**	§6
8	**Add context, don't widen the limit.** Eclipse and rain are causes, not faults	§6
9	**Write the rules and settings down first. Never tune on the test**	§7 §9
10	**Split by time, leave an embargo, and `assert` it**	§9
11	**Baselines first. Same period, same scorer, every method. Beat the best one**	§9 §10
12	**Recall and precision, never accuracy, for rare events**	§7
13	**Explain every flag.** Right answer, right reason	§8
14	**One split is one result.** Backtest on many and report the spread	§10
15	**A range, checked.** Bands from past misses with their coverage; remaining life as a window, never a single day	§10 §11
16	**If the simple method wins or ties, recommend it.** That's a professional result	§9
17	**Four sentences, one number, one action.** Then stop talking	§12

1. Colab habits (every day)

Rule	Why	Taught
Save a copy in Drive first	The original can't be saved. Your work would vanish	W1D1
Run cells top to bottom, one at a time	Each cell uses what the cells above created	W1D1 W1D3
Shift + Enter runs a cell and jumps to the next	Fastest way through the notebook	W1D1
Replace every `___` with your answer, then run that cell	The check only marks filled-in answers	W1D1
After any restart, run STEP 0 again	A restart wipes Colab's memory	W1D1
Text goes in quotes (`"timestamp"` , `"W-MON"` , `"2025-09-01"`). Numbers, `True` / `False` and variable names don't	Without quotes, Python thinks text is a variable name. With quotes, `"True"` is just text	W1D1 W1D3
Never paste real NIGCOMSAT data into the AI (Gemini) button	Course data is synthetic and safe. Real telemetry is restricted	W1D1

Everything is weird? Runtime → Restart session and run all.

Stuck? Climb the ladder.

1. **Read the ❌ hint**: it usually says exactly what's wrong.
2. **Check the error table** at the end of this book (or today's rule book).
3. **Ask your neighbour.** Explaining helps both of you.
4. **Ask the AI button** to *explain the error*. Paste your code and the error, never real data.
5. **Raise your hand.**

2. Load and clean the data

```
df = pd.read_csv(data_path("FILE_NAME"),
                 parse_dates=["TIME_COLUMN"],   # 1. text → real dates
                 index_col="TIME_COLUMN")       # 2. time becomes the row label
df = df.asfreq("D")                             # 3. declare the rhythm: "h" hourly, "D"
daily, "W-MON" weekly, "MS" monthly
```

Rule	Why	Taught
Declare the rhythm with `.asfreq()` : `"h"` , `"D"` , `"W-MON"` , `"MS"`	It reveals the hidden missing rows. Capital `"H"` crashes on pandas 3; plain `"W"` means weeks ending on **Sunday**	W1D1 W5D2 W6D3

Rule	Why	Taught
Load the right file. `battery_health_daily` exists in week-03 (clean) AND week-04 (planted faults)	Same name, different data. The wrong one gives the wrong story	W2D3
Profile before you plot	Gaps and duplicates silently break everything after	W2D3
One time zone, written down. Telemetry usually records in UTC; Lagos (WAT) = UTC + 1	"21:00" means nothing until you say whose clock	W1D1 W6D3
Clean, then say what you cleaned	Every gap filled is an assumption written down	W6D3

3. Charts that don't lie

Rule	In code	Taught
Draw the threshold (the line that matters: 0 dB, the SLA %)	`plt.axhline(0, color="red", ls="--")`	W1D2 W2D3
Title = the finding, with a number	`plt.title("Link went down on 11 of 31 days")`	W1D2 W2D3
Sensible axis: counts start at 0; a bounded % near the top may zoom in, but **say the range**	`plt.ylim(0, None)` or `plt.ylim(96, 100)`	W1D2
One y-axis per chart. Two units? Two stacked charts	`plt.subplots(2, 1, sharex=True)`	W1D2
Units on the axis	`plt.ylabel("link margin (dB)")`	W1D2
Compare groups on one shared y-axis	`wide.plot(subplots=True, sharey=True)`	W1D2

Pick the right daily summary W1D2

Your question	Use	Why
"Did it ever go down?" / SLA breach	`.resample("D").min()`	the **worst moment** of each day
"Did it ever overheat?"	`.resample("D").max()`	the worst moment, other direction
"What's the typical level?" / trend	`.resample("D").mean()`	averages smooth away short events

4. Smoothing with rolling windows

Rule	Why	Taught
Roll one column, not the whole table: `tk["tickets_raised"].rolling(7)`	The column is the series. The table has other columns too	W1D3
A window is a number of rows: `rolling(7)` on daily data = 7 days	On hourly data, `rolling(24)` = one day	W1D3
The first N−1 values are `NaN`	A 7-day average doesn't exist until day 7. It's not an error	W1D3
Trailing (the default) for anything live	`center=True` peeks at future days: reports only	W1D3 W2D3
Never report an SLA from a smoothed line	Short outages disappear inside the average	W1D3
Pick a window from the data: daily tickets → 7; rare counts (P1) → 30; hourly → 24	Long enough to calm the noise, short enough to see change	W1D3

5. Season and memory

Decomposition

Rule	Why	Taught
observed = trend + seasonal + residual	That's the additive model. The three parts add back to the original exactly	W2D1
The period comes from the process. Ask: "what makes this repeat, and how many rows until it repeats?"	Hourly + daily eclipse → 24. Daily + working week → 7. Monthly + rainy season → 12	W2D1 W2D3
Swing grows with the level? Multiply	Use `model="multiplicative"`, or take `np.log()` first and stay additive	W2D1
Three full cycles minimum	Fewer, and the "season" might be one lucky year	W2D1
Look for trouble in the residual	Trend and season are explained. The leftovers are the surprises	W2D1
One disaster or a moving season? Use `STL(robust=True)`	It stops one bad month from spoiling every month like it	W2D1

Autocorrelation

Rule	Why	Taught
A lag is a shift. `s.shift(1)` = yesterday, `s.shift(7)` = same weekday last week	The copy must line up with the original, row by row	W2D2
Autocorrelation goes on ONE column: `s.autocorr(lag=7)`	A whole table (DataFrame) has no `.autocorr`	W2D2
Always `.dropna()` before `plot_acf`	One empty value breaks the correlogram	W2D2
Ignore the bar at lag 0	It is always 1: the series compared with itself	W2D2
The period is the GAP between spikes, in the data's own units	Hourly data: lag 24 = one day. Daily data: lag 7 = one week	W2D2
Two trending series? Compare `.diff()` before you believe them	A shared trend makes unrelated things look linked	W2D2
The correlogram confirms; it doesn't guess for you	Measure the repeat, don't assume it	W2D3

Reading a correlogram W2D2

You see	It means
Tall spikes at 7, 14, 21…	A weekly season, **period 7**
Tall spikes at 24, 48… (hourly data)	A daily season, **period 24**
High bars that fade slowly	A **trend**
Everything inside the shaded band	**No memory**: noise

6. Alarms, control charts and CUSUM

Building an alarm

Rule	Why	Taught
An alarm is a True/False column: `alarm = bus["col"] < 20`	Then `.sum()` counts how many times it fires	W3D1
Combine conditions with `&`, and wrap each one in brackets: `(a < 20) & (b == 0)`	Python's `and` does not work on columns	W3D1
`~` **means "not"**: `hot & ~after_eclipse`	Hot, but not right after an eclipse	W3D1
The flag is a number, not text: `== 0`, not `== "0"`	`eclipse_flag` holds 0 and 1	W3D1
Add context, don't widen the limit. Condition on known causes (eclipse, rain)	A wider limit also hides the real faults	W3D1 W4D1

Rule	Why	Taught
Always ask "where did that number come from?"	A limit with no reason is a guess	W3D1
Rain is not a fault. Margin below 0 dB is	A detector must separate "raining" from "link down"	W3D1

Control charts

Rule	Why	Taught
Limits come from a quiet reference period, never from the whole file	The whole file contains the faults you're trying to catch	W3D2
Control limits = centre ± 3σ (not min/max, not design limits)	3σ keeps false alarms to about 8 per 2,880 normal hours	W3D2
One set of limits per operating state (eclipse, recharge, steady sun)	Mixing states makes σ 8× bigger and the chart goes blind	W3D2 W4D2
Trailing windows only, with `.shift(1)`	A reading can't help judge itself	W3D2
Robust = median + MAD × 1.4826	One wild hour can't poison the ruler	W3D2
Residual = observed − expected, in that order	Positive = hotter / higher than expected	W3D2
Check the residual is clean before you trust it	Mean ≈ 0, no autocorrelation left, constant spread	W3D2

Three states, three normals W4D2

State	When	Normal looks like
🌘 Eclipse	21:00 UTC, in the eclipse seasons	array ≈ 0 A, bus sags, battery discharging
🔋 Recharge	the 3 hours after an eclipse	array ≈ 29 A, battery warm, charge climbing
🌞 Steady sun	everything else	array ≈ 28 A, charge 100%, bus ≈ 50.4 V

Slow change: drift and CUSUM

Rule	Why	Taught
Name the shape first: spike, level shift or drift	Each shape needs a different tool	W3D3
Take away what you already know (season, drive level, ageing) before you watch	Otherwise the normal yearly swing hides the fault	W3D3
The CUSUM line: `S = max(0, S + leftover − k)`	Add today, minus the slack, never below zero	W3D3
Set k and h from σ: `k = 0.5 * sigma`, `h = 5 * sigma`	σ = the normal wobble of the leftovers in a calm period	W3D3

Rule	Why	Taught
Alarm date ≠ start date. Start = the day after S was last 0	CUSUM needs time to collect evidence	W3D3
Drift? Watch the slope, not the level	The battery never crosses 95 Ah, but its fade gets 1.6× faster	W3D3

7. Scoring a detector

The 4 scoring rules (write them down BEFORE you score) W4D1

#	Rule
1	One flag anywhere inside an event's start→end (±1 time step) = **caught**
2	Many flags inside one event = still **one** catch
3	A flag outside every event = **one false alarm**
4	A long event is still **one** event

The 3 numbers W4D1

Number	Formula	Plain question	Use it?
Recall	caught ÷ real events	Did we miss anything?	✅ always
Precision	caught ÷ (caught + false alarms)	When it pages, is it real?	✅ always
Accuracy	right answers ÷ all readings	–	❌ **never** for rare events (doing nothing scores 99.87%)

Rule	Why	Taught
Write the rules first, then compute. Never change them after seeing the score	Otherwise you are grading your own exam	W4D1
Never report accuracy for rare events. Report recall **and** precision	Doing nothing looks excellent on accuracy	W4D1 W6D3
Decide which number leads before you score	Recall when a miss could cost the satellite; precision when too many pages make people stop reading them	W4D1
Say false alarms in hours	That's the number a manager can act on	W4D1
Always look at `event_table`, not just the score	A long event can be "caught" by a flag that has nothing to do with it: a lucky catch	W4D2

8. Machine learning with scikit-learn

The pattern (every model, every time) W4D2

Step	Code	In plain words
1. Build X	`X = bus[CHANNELS]`	A table of numbers: **rows = hours**, **columns = channels**
2. Make a model	`IsolationForest(n_estimators=200, contamination=0.005, random_state=42)`	200 random trees · allowed to flag 0.5% · same result every run
3. Fit	`model.fit(X[train])`	Learn what "normal" looks like from the training months. **No labels**
4. Ask	`model.predict(X)`	**-1 = odd**, 1 = looks normal
5. Score	`model.decision_function(X)`	A number per hour: **lower = more odd**

The three kinds of eyes W4D3

Eye	The question it answers	Example
🕰️ **History**	What was it an hour ago? Over the last day?	`soc.shift(1)` , `soc.rolling(24).mean()`
🌒 **Context**	Which state is the satellite in?	`steady_sun(bus)` : no eclipse this hour or the 3 before
🔗 **Relationship**	Is it normal **for** the other sensor?	`temp_per_drive = twta_temp_c / rf_drive_pct`

Rule	Why	Taught
Sanity-check the count before you look at the results: contamination × rows ≈ flags	Thousands of flags = a bug	W4D2
Rare is not broken	A forest finds what is unusual; eclipse is unusual and perfectly normal	W4D2
Give the model context (steady sun)	Don't hope it works it out	W4D2
The model cannot tell time. If history matters, put it in a column	It sees one row at a time	W4D3
Model the relationship, not the raw value, when the raw value swings with the load or the season	"Hot for its workload" is the fault; "hot" alone may just be a busy hour	W4D3
A big σ on the fault is not the test. The test: does the feature stay quiet on normal months it never saw?	A feature that fires everywhere catches nothing	W4D3

Rule	Why	Taught
Fewer, sharper features. Prune twins above 0.98	Every column should answer an engineer's question	W4D3
Check *why* it flagged: which hours, what time of day, which channel	A catch for the wrong reason is a lucky catch	W4D2 W5D1
Explain every flag with per-channel z-scores: `z = (X - X[ref].mean()) / X[ref].std()`	The biggest |z| is your sentence for the duty engineer: *"temp_per_drive is 3.8σ high"*	W4D2 W4D3
Scale before distance methods (k-means)	The forest doesn't need it, but the pipeline habit never hurts	W4D2
Set `random_state`	Same result tomorrow, and the same as your neighbour's	W4D2
The old chart still wins some fights	A single-channel spike is the chart's home ground	W4D2

9. Honest testing: no expo

What expo looks like in time series W5D1

Expo	What it looks like	The honest version
👥 **Neighbours in training**	A random split: the 10:00 test hour has 09:00 and 11:00 in training	Split **by time**: past to train, future to test
🔮 **Features that peek ahead**	`shift(-1)`, `rolling(..., center=True)`	Only `shift(1)`, `shift(24)`, plain `rolling(24)`
🔑 **Settings tuned on the key**	Picking `k` or `contamination` by looking at the test answers	Fix settings first, or tune on the **training** months only

The leak test (ask it of every feature) W4D3 W5D3

> ***"Could I compute this at 14:00, using only what I knew at 14:00?"*** For weekly forecasts: *"…on Monday morning, before this week's tickets arrive?"* If not, it's a leak. Delete it.

✅ Honest (past only)	❌ Cheats (peeks at the future)
`x.shift(1)`, `x.shift(24)`, `x.shift(52)`, `x.diff(1)`	`x.shift(-1)`: copies the NEXT row back (tomorrow's newspaper)
`x.rolling(24).mean()`, `.std()`	`x.rolling(24, center=True)`: mixes in future rows
Things known in advance: the rollout plan, the calendar (`sin52`, `cos52`)	This week's own count as a feature

✅ Honest (past only)	❌ Cheats (peeks at the future)
Anything fitted on the training rows only	Anything fitted on the whole year (or the test weeks), then judged on them

Rule	Why	Taught
Every feature is known before the moment you judge. `shift(+n)` only	Otherwise the model reads tomorrow's newspaper	W4D3 W5D3 W6D3
Never shuffle a time series before splitting. Train on the past, test on the future	A random split puts the test hour's neighbours in training	W5D1 W6D3
Embargo ≥ your longest feature window. Then `assert` the split	So no feature on the first test hour can reach back into training	W5D1 W6D3
Fix every setting before you look at the test (k, contamination, α, z, windows, folds). Tune only on the training months	Settings chosen on the test are expo	W4D1 W5D1 W5D2 W5D3 W6D1
Same months, same hours, same scorer for every method you compare, baselines included	Otherwise you're comparing different exams	W4D1 W5D1 W5D2 W6D1 W6D3
Compare like with like: same weeks, same horizon, same information	"1 week ahead" (it saw last week's real count) is easier than "8 weeks, blind"	W5D3
Write down your reference period and why operations believe it was quiet	Your limits and your model learned "normal" from it	W5D1
If the simple method wins or ties, recommend it	That's a professional result, not a failure	W5D1 W6D1 W6D3

10. Forecasting

The baselines W5D2

Baseline	Grandma says	One line
🐵 **Naive**	"Same as last week"	`series.iloc[-1]`
📅 **Seasonal naive**	"Same week last year"	`series.iloc[-52:]`
🧱 **Moving average**	"The average of the last 8 weeks"	`series.iloc[-8:].mean()`
🍵 **Exponential smoothing**	"A running level"	`alpha * v + (1 - alpha) * level`
🎯 **Rate seasonal naive**	"Last year's rate × today's network"	`rate = series / terminals`

MAE (mean absolute error) = the average size of the misses. *"On a typical week we were off by N tickets."* Lower is better.

Rule	Why	Taught
Baselines first. Before any model, forecast with the cheap ones	A model that can't beat them has not earned its place	W5D2
Beat the BEST baseline, not the one with the fanciest name	Beating the weakest proves nothing	W5D2 W5D3
Lock the test weeks away and forecast them blind, from one origin	No peeking, no expo	W5D2
Say MAE in words: "off by about N tickets on a typical week"	A manager can act on that sentence	W5D2
Divide by the size you already know (terminals, drive, rooms)	A rate repeats; a count grows	W5D2
Read the coefficient signs	A sign that makes no physical sense is a bug until proven otherwise	W5D3
One split is one result, one screenshot. Never trust a single held-out score on its own	Another stretch of weeks can give another verdict	W5D2 W6D1
Only the past at each origin, and refit every time	That's what makes it a backtest	W6D1
Report the spread across folds, and **when** the bad folds happen	The average hides the season changes	W6D1
Test at the horizon the user needs	A forecast that needs last week's count can't make a month-ahead roster	W6D1
A band is a promise. Build it from past misses that include the hard weeks, then check its coverage	A band nobody checked is a guess with decimals	W5D3 W6D3
Say it with a range and name the thing that could blow it up	"Most likely 56, plausibly 49 to 63; expect bigger misses at season changes"	W5D3

The honest band recipe W5D3

1. Walk forward through the past: fit on the weeks **before** each week, forecast it, write down the miss.
2. Make sure those weeks include the **hard** weeks you'll forecast (season changes!).
3. Half-width = the **90th percentile** of the absolute misses: `np.quantile(np.abs(misses), 0.9)`.
4. Band = forecast ± half-width. **Check coverage** on weeks it never saw: about 7 or 8 of 8 for a 90% band.
5. Never pick the width by looking at the test weeks.

Backtest words W6D1

Word	What it means
Origin	The week you are "standing in" when you forecast. You may only use weeks **before** it

Word	What it means
Horizon	How many weeks ahead you forecast: 1 (next week), 4 (the month-ahead roster), 8
Fold	One exam: train on the past, forecast a block of weeks, score it
Walk forward	Refit at each cut, forecast the next block, score, slide on. The training block only grows
Rolling origin	Stand at **every** week, forecast `h` weeks blind, score, move one week on
Blind	The whole block is forecast at the origin; no peeking at any week inside it

11. Remaining useful life

The limits are **known up front** (`degradation_thresholds.csv`): battery capacity 80 %, amplifier gain −2 dB, propellant 5 kg. We forecast *when* they are reached. **Remaining useful life** = (what's left above the limit) ÷ (how fast it's going).

Rule	Why	Taught
Fit on the past only	Make the call with the data you'd have had on that day	W6D2
"No crossing" is an answer. Never let a function turn "never" into day 0. Check for `None`	"Not inside the window" is information too	W6D2
Sanity-check with a simple sum: left above the limit ÷ rate per year	If it disagrees with the model, one of them has a bug	W6D2
Define "crossed" before you score: first dip, 30-day mean or last good day	Write down which, and why	W6D2
Backtest the method from several starting points	Note which way it errs: early or late	W6D2
Report a window, never a single day, and say how far past the data you reached	The further past the data, the less sure the date	W6D2 W6D3
Act before the crossing, and re-estimate every time new data arrives	Each new month of data can move the window	W6D2

12. Telling the story, the project and demo day

The 4-sentence finding W1D2 W2D3 W6D3

Sentence	Answers	Must have
Situation	What's going on?	the process and who runs it
Observation	What did the data show?	**a number**
Implication	So what?	the operational meaning
Recommendation	What should we do?	an action someone can take. **Always last**

Rule	Why	Taught
Four sentences, one number, one action. Then stop talking	It's the sentence your unit head repeats	W6D3
Name the process and its owner	A tool without an owner is a demo, not a deployment	W6D3
Say one limitation out loud	Unit heads trust the engineer who knows where the tool stops	W6D3

Demo-day rules W6D3

#	Rule	Why
1	**5 minutes, then 1 question.** The timer is the MC	Everyone presents inside the hour
2	**Six beats:** problem 0:45 · data 0:45 · method 0:45 · honest test 1:00 · the tool live 1:00 · recommendation 0:45	The honest test and the tool carry the most marks
3	**Show the tool live** from the training server, with a screenshot as backup	"Deployed" means it runs where your unit can use it
4	**One table** for the honest test: your method vs the baseline, same period, same scorer	Judges can't read three tables in a minute
5	**End on the recommendation**, not on "any questions?"	It's the sentence your unit head repeats
6	**Say one limitation out loud**	Trust comes from knowing where the tool stops
7	**No real data on the projector** if outsiders are present	Show charts and summaries, not raw rows
8	**Two stars and a wish** for every other presenter	Kind, specific, about the work

Quiz rules (Quizzes 1–4, 20 minutes) W2D3

☐ Phones face down, notebooks closed, **no AI tools**.

☐ One answer per question. True/False questions have two options.

☐ Read the whole question; don't hunt for patterns in the letters.

☐ We mark it together. Wrong answers are where you learn the most.

Final pre-submission checklist W6D3

☐ **Agreement:** my unit head's email names the process, the data and who the tool is for. It's in my submission.

☐ **Hygiene:** DatetimeIndex, declared frequency, one time zone (written down), gaps and duplicates handled and **explained**.

☐ **Honest chart:** sensible axis, the threshold drawn, a title that states the finding.

☐ **Season:** the period comes from a correlogram, and I decomposed with it.

☐ **Leak test:** every feature passes. Trailing windows and positive shifts only.

☐ **Split by time** with an `assert` and an embargo ≥ my longest feature window.

☐ **Baseline** on the same period, with the same scorer.

☐ **Spread / repeated calls / coverage** reported (folds for forecasts, several origins for remaining life, coverage for bands).

☐ Detection: **event-level** precision and recall. **No accuracy figure.**

☐ Remaining life: a **window**, never a single date.

☐ If the simple method won or tied, I **recommend it**.

☐ **Deployed** on the training server (dashboard or scheduled report). The README says how to run it.

☐ Repository: code + `requirements.txt` + `README.md`. **No raw NIGCOMSAT data.**

☐ Report: 2–4 pages, the 4 sentences, 3 limitations, the AI-use note.

☐ Demo rehearsed once with a timer: under 5:00.

The W6D3 notebook's **self-check toolkit** (`hygiene`, `leak_test`, `check_temporal_split`, `compare`, `coverage`, `recommendation`) runs most of these checks for you. Copy it into your project.

13. When things go wrong: error → fix

The common errors from all 18 days. The daily rule books also cover the errors specific to each exercise task.

Notebook and blanks

You see	It means	Fix
`✍️ Fill in the blank first` / `AssertionError: ✍️ Fill in … first`	A `___` is still there	Replace it, run the cell again
`AssertionError: ✍️ Finish … first` / `✍️ Do … first`	You skipped an earlier task	Do that task, then come back
`❌ Not yet … (Python said: TypeError)` under a function	The `___` inside the `def` is still there	Fill the blank in the `def` line or the body, run the cell again
`NameError: name 'check' is not defined`	STEP 0 was not run	Run STEP 0
`NameError` for something you know exists (`ka`, `score_events`, `make_features` …)	Colab forgot (restart), or you skipped a ▶️ or 🧰 tool cell	Run STEP 0, then the cells above, top to bottom
`TypeError: … 'NoneType'`	A variable above is still blank, or a function answered `None`	Fill the blank above. A `None` from `day_of_threshold` means "no crossing": that's an answer
A chart is empty or doesn't show	The cells above didn't run, or this one didn't finish	Runtime → Run before, then run it again; end with `plt.show()`
"This notebook was not authored by Google"	Normal warning	Click **Run anyway**
Everything is weird	–	**Runtime → Restart session and run all**

Quotes, names and typos

You see	It means	Fix
`NameError: name 'timestamp' is not defined` (or `'W'`, `'target'`, `'tickets_lag1'`)	Text without quotes	Column names, dates and frequencies are text: `"timestamp"`, `"W-MON"`
`AssertionError: ✍️ Use the variable itself, without quotes`	You typed `"steady"`	Variables have no quotes: `steady`
A ❌ on a `True` / `False` setting	You typed `"True"`	`CENTER = True`, no quotes

You see	It means	Fix
`ValueError: window must be an integer` / `TypeError: unsupported operand…`	You did maths with text (`"7"`)	Numbers have no quotes: `WINDOW = 7`
`SyntaxError: leading zeros in decimal integer literals`	A date without quotes	Dates are text: `"2025-09-01"`
`SyntaxError`	A typo: missing `)` , `"` or `,`	Compare with the recipe, character by character
`KeyError: 'column_name'`	Wrong column name, spelling or table	Copy the name exactly from `.head()`
A count of `0` that should not be 0	You compared a number with text (`== "0"`)	Use `== 0`

pandas and time

You see	It means	Fix
`ValueError: Invalid frequency: H`	Old capital code; pandas 3 removed it	Use `"h"` (and `"1h"` for a tolerance); `"D"` , `"W-MON"` , `"MS"`
Weekly data shows hundreds of gaps	You used `"W"` (weeks ending on **Sunday**)	Every date is a Monday: `"W-MON"`
`ValueError: cannot reindex on an axis with duplicate labels`	`.asfreq()` on data with a repeated timestamp	Drop duplicates first: `df[~df.index.duplicated()]`
`ValueError: Index contains duplicate entries, cannot reshape`	Wrong columns in `pivot`	index = the time, columns = the group
`KeyError: 'timestamp'` on a new file	This file's time column has another name	Read the CSV header (e.g. `date` , `week_start`)
`IndexError: index 0 is out of bounds`	Your filter matched nothing	Check the `<` direction and the value
A smoothed line starts late	Normal: the first `WINDOW - 1` values are `NaN`	Use `.dropna()` before `.iloc[0]`
`ParserError: Expected 8 fields …` on the anomaly labels	Plain `pd.read_csv` on the answer key	Use `load_labels()` : it repairs the stray comma

You see	It means	Fix
Battery data looks different from your neighbour's	Wrong file version	`data_path("battery_health_daily")` gives week-04 (faults); week-03 is the clean one

Alarms, decomposition and correlograms

You see	It means	Fix
`ValueError: The truth value of a Series is ambiguous`	You used `and` / `or` between columns	Use `&` / `|` and brackets: `(a < 20) & (b == 0)`
`TypeError: unsupported operand type(s) for &`	Missing brackets around a comparison	`(bus["eclipse_flag"] == 0)` in brackets
CUSUM alarms on day 1	You forgot `- k`, or used `h` where `k` belongs	k = 0.5σ goes **inside** the loop; h is only the alarm line
`AttributeError: 'DataFrame' object has no attribute 'autocorr'`	You used the whole table	Pick one column: `df["col"].autocorr(lag=1)`
`MissingDataError` / `exog contains inf or nans`	There's a `NaN` in the series	Add `.dropna()` before `plot_acf`
`ValueError: This function does not handle missing values`	There's a `NaN` inside the series	Find it with `.isna().sum()`; fill or trim it first
`ValueError: x must have 2 complete cycles…`	The period is too big for the data	Use more data, or check the period is right
`ValueError: Multiplicative seasonality is not appropriate for zero and negative values`	Multiplicative needs all values above zero	Use additive, or check for zeros
`ValueError: You must specify a period…`	No `period=` and no frequency on the index	Add `period=` (or `.asfreq()` the series)
Trend panel empty at both ends	Normal: a centred moving average needs data on each side	Nothing to fix
Picture shows twice	The plot was returned **and** shown	`fig = parts.plot()`, then `plt.show()`

Scoring and scikit-learn

You see	It means	Fix
`{'events': 0, …}` / `ZeroDivisionError` from the scorer	The file name doesn't match the answer key	Copy it exactly, with `.csv`: `"bus_power_telemetry_hourly.csv"`

You see	It means	Fix
False alarms look huge	You scored eclipse hours as faults	Judge the right state: `& (bus["eclipse_flag"] == 0)`, or `steady`
Thousands of flagged hours	You flagged the **normal** hours, or wrote contamination as a percentage	Odd is `-1`, not `1`; contamination is `0.005`, not `5` or `0.5`
`InvalidParameterError: … contamination`	contamination isn't a small fraction	Use `0.005` (that's 0.5%)
Forest numbers differ a little from your neighbour's	Different `random_state` or library version	Use `random_state=42`. Small differences are normal; the checks allow a range
`TypeError: '<' not supported …` with a date	The date is a number	Write it as text: `"2025-09-01"`
`AssertionError: leakage: training overlaps the test`	Your dates are the wrong way round	`TRAIN_END` must come **before** `TEST_START`
k-means gives a strange answer	No scaler	`StandardScaler()` (with the brackets) first

Forecasting and remaining life

You see	It means	Fix
A leak warning ("tomorrow's newspaper")	You typed a negative shift	A positive number looks back: `shift(1)`
`ValueError: Quantiles must be in the range [0, 1]`	You wrote `90`	Write `0.9`
`ValueError: operands could not be broadcast together …`	The forecast and the test block have different lengths	Forecast exactly the weeks you score
❌ `not a split by time: training reaches …`	The split is shuffled, or the dates are the wrong way round	Train = **before** the cut, test = **from** the cut
❌ `… forecasts but the test has … weeks`	Two methods were scored on different periods	Forecast the same weeks for every method
A crossing date one day off from your neighbour's	One used the formula (rounds down), the other `day_of_threshold`	Same line, same answer: a day either way doesn't matter for a multi-year call
A backtest cell takes a few seconds	It runs hundreds of small forecasts	Normal. Wait for the table

Demo day

You see	Fix
The dashboard won't load (server slot, VPN or port)	Switch to your screenshot and keep talking; fix it after
You're at 4:30 and not at the recommendation	Skip to the 4 sentences. Always